Step 5 — An Approval Agent That Only Asks a Human When It Really Needs To — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 5: asking a person about **every** answer (Steps 2-4) is safe but too slow — nobody keeps a tool that stops on every question. The Approval agent now runs an explicit checklist on each answer: grounded? cites a source? asks for an action? If every check passes, it approves by itself. If any fails, it pauses for a human and says **which** check failed. **If not:** you'd have to choose between "a human checks everything" and "nobody checks anything".

After this step the project is complete. Only `agents/approval_agent.py` and the tests change — the graph, `main.py` and the pause/resume scripts already work with it.

Solution — an explicit checklist, and a human only when a check fails

Changed from Step 2: `approval_node` keeps its name, its pause and its yes/no handling. Two things change: the one fixed line of `reasons` becomes `run_checks(state)`, and an "all passed → approve" block is added before the pause. The check functions above it are new.

Story — `agents/approval_agent.py`: each check is its own small function, and `run_checks()` returns the list of checks that **failed**. An empty list means auto-approve. Otherwise `interrupt()` pauses with the answer and the failed checks, so the person knows *why* they're being asked. **If not:** one vague "is this okay?" model call couldn't tell you which rule a wrongly-approved answer slipped past.

```
# src/documind/agents/approval_agent.py
from langgraph.types import interrupt

from documind.state import DocState
from documind.utils.logger import get_logger

# why: tasks asking the system to DO something are always checked by a
# person — a wrong answer is bad, a wrong action is worse
# new in Step 5: the checklist, one small function per check
ACTION_WORDS = ["send", "cancel", "delete", "refund my", "change my"]
```

```
def cites_a_source(answer: str, sources: list[dict]) -> bool:
    for source in sources:
        if "[" + source["source_file"] + "]" in answer:
            return True
    return False

def asks_for_an_action(task: str) -> bool:
    lowered = task.lower()
    for word in ACTION_WORDS:
        if word in lowered:
            return True
    return False

def run_checks(state: DocState) -> list[str]:
    # why: an explicit checklist, one item at a time — so a log line can
    # say exactly WHICH check sent an answer to a human
    failed = []
    if not state["grounded"]:
        failed.append("not grounded in the documents")
    if not cites_a_source(state["answer"], state["sources"]):
        failed.append("cites no source file")
    if asks_for_an_action(state["task"]):
        failed.append("the task asks for an action")
    return failed

def approval_node(state: DocState) -> dict:
    logger = get_logger(__name__)
    # changed in Step 5: real checks replace the fixed reason
    reasons = run_checks(state)

    # new in Step 5: nothing failed -> approve without asking
    if not reasons:
        # when: every check passed — approve without bothering a person
        logger.info("auto-approved")
        return {"approved": True, "path": ["auto_approved"]}

    logger.info("asking a human, failed checks: %s", reasons)
    # how: interrupt() pauses the graph here and saves the state; the
    # value passed to Command(resume=...) comes back as `decision`
    # when: on resume, this node runs again from its first line — so the
    # log line above prints twice, and nothing above interrupt() may do a
    # real action (like sending an email)
    decision = interrupt({"answer": state["answer"], "failed_checks": reasons})
    if decision == "yes":
        return {"approved": True, "path": ["human_approved"]}
    return {"approved": False,
```

```
            "answer": "Not approved — nothing was sent.",
            "path": ["human_rejected"]}
```

`main.py` doesn't change: since Step 2, `ask_human()` already prints the answer and the reasons it receives.

Story — `tests/unit/test_approval_checks.py`: each checklist item, on hand-made states. **If not:** a broken check (say, citations never matching) would send every answer to a human, or none.

```
# tests/unit/test_approval_checks.py
# Checks the Approval agent's checklist with hand-made states.
# No API key needed.
# Run from the repo root: python tests/unit/test_approval_checks.py
from documind.agents.approval_agent import run_checks

SOURCES = [{"source_file": "refund_policy.md"}]

def make_state(task: str, answer: str, grounded: bool) -> dict:
    return {"task": task, "answer": answer, "grounded": grounded,
            "sources": SOURCES}

def test_all_checks_pass() -> None:
    state = make_state("What is the refund policy?",
                       "30 days [refund_policy.md]", True)
    print(f"case 1 (grounded, cited)   -> {run_checks(state)}")

def test_no_citation() -> None:
    state = make_state("What is the refund policy?", "30 days", True)
    print(f"case 2 (no citation)       -> {run_checks(state)}")

def test_action_request() -> None:
    state = make_state("Send the refund policy to the customer",
                       "30 days [refund_policy.md]", True)
    print(f"case 3 (asks for action)   -> {run_checks(state)}")

if __name__ == "__main__":
    test_all_checks_pass()
    test_no_citation()
    test_action_request()
```

Changed from Step 4: the integration test now expects case 1 to be **auto-approved**, and adds case 4 and case 5 — an action request that pauses, approved in one run and rejected in another.

```
# tests/integration/test_live_graph.py
# Runs the whole graph against the real OpenAI API.
```

```
# Run from the repo root: python tests/integration/test_live_graph.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from documind.config import load_config
from documind.graph import build_graph
from documind.state import start_state

def run(graph, thread_id: str, task: str) -> dict:
    config = {"configurable": {"thread_id": thread_id}}
    return graph.invoke(start_state(task), config)

def test_grounded_answer(graph) -> None:
    # case 1: a document question — searched, answered, auto-approved
    result = run(graph, "t1", "What is the refund policy?")
    print(f"case 1 (grounded)     -> {result['path']}")
    print(f"                         {result['answer']}")

def test_not_in_documents(graph) -> None:
    # case 2: searched, but nothing relevant — must not make one up
    result = run(graph, "t2", "What is the policy on remote work?")
    print(f"case 2 (not found)    -> {result['path']}")

def test_general_question(graph) -> None:
    # case 3: not about the documents — no search at all
    result = run(graph, "t3", "What is the capital of France?")
    print(f"case 3 (general)      -> {result['path']}")

def test_action_pauses(graph) -> None:
    # case 4: asks for an action — must pause for a human, both answers
    task = "What is the refund policy? Send it to the customer."
    paused = run(graph, "t4", task)
    print(f"case 4 (paused?)      -> {'__interrupt__' in paused}")
    config = {"configurable": {"thread_id": "t4"}}
    approved = graph.invoke(Command(resume="yes"), config)
    print(f"case 4 (human: yes)   -> approved={approved['approved']}")

    run(graph, "t5", task)
    config = {"configurable": {"thread_id": "t5"}}
    rejected = graph.invoke(Command(resume="no"), config)
    print(f"case 5 (human: no)    -> {rejected['answer']}")

if __name__ == "__main__":
    load_config()
    # how: tests use a fresh MemorySaver — nothing is written to disk
    graph = build_graph(MemorySaver())
```

```
    test_grounded_answer(graph)
    test_not_in_documents(graph)
    test_general_question(graph)
    test_action_pauses(graph)
```

Run everything:

```
python tests/unit/test_chunking.py
python tests/unit/test_routing.py
python tests/unit/test_approval_checks.py
python tests/integration/test_live_graph.py
python scripts/pause_demo.py
python scripts/resume_demo.py
python -m documind.main
```

Expected integration output:

```
case 1 (grounded)     -> ['clean_task', 'retrieve_node', 'reason_node',
                          'auto_approved']
case 2 (not found)    -> ['clean_task', 'retrieve_node', 'no_grounding_node']
case 3 (general)      -> ['clean_task', 'answer_node']
case 4 (paused?)      -> True
case 4 (human: yes)   -> approved=True
case 5 (human: no)    -> Not approved — nothing was sent.
```

Expected app session (log lines left out; wording varies):

```
You: What is the refund policy?
  path: clean_task -> retrieve_node -> reason_node -> auto_approved
Answer: Customers can return any product within 30 days of purchase
for a full refund [refund_policy.md].
You: Send the warranty terms to the customer.
  -- needs your approval --
  answer: Every product has a 1-year warranty against manufacturing
faults [warranty.md].
  why:   the task asks for an action
  Approve? (yes/no): yes
  path: clean_task -> retrieve_node -> reason_node -> human_approved
```